

UNIT 4

FUNCTIONS

1.Introduction

A Function is a reusable block of code designed to perform a specific task. It helps break large programs into smaller, logical parts. Functions make code cleaner, easier to understand, and more maintainable.

C++ functions can take inputs (called parameters), execute a block of statements, and optionally return a result. C++ also supports advanced features like function overloading, default arguments, and inline functions.

```
#include <iostream>
using namespace std;
// Void function definition
void hello() {
    cout << "Hello World" << endl;
}
// Return-type function definition
int square(int x) {
    return x * x;
}
int main() {
    // Calling the void function
    hello();
    // Calling the return-type function
    int result = square(5);
    cout << "Square of 5 is: " << result << endl;
    return 0;
}
```

There are three functions in above C++ program:

- I. **main() function:** This is the entry point of every C++ program. When the program runs, execution starts from the main function. It contains the calls to the other functions.
- II. **hello() function:** This is a user-defined function that does not take any parameters and does not return a value. It simply prints "Hello World" to the console. The return type void indicates that it doesn't return anything.
- III. **square(int x) function:** This is another user-defined function that takes an integer as input and returns the square of that number. The return type int means the function will return an integer value. In main(), the function is called with the value 5, and the returned result is stored in a variable and printed.

1.1 Function Syntax

A function in C++ follows this general format:

```
return_type function_name(parameter_list) {  
  
    // body of the function  
  
}
```

Each part has a specific role:

- I. Return type: Specifies what type of value the function returns. Use void if there is no return value.
- II. Function name: The name you will use to call the function.
- III. Parameter list: Inputs that the function accepts. It can be empty if no inputs are needed.
- IV. Function body: The block of code that runs when the function is called.

1.2 Function Declaration vs Definition

1.2.1 Function Declaration

Introduces the function to the compiler. It tells the compiler the return type, name, and parameters but does not include the body. It ends with a semicolon.

```
// Declaration  
int add(int, int);
```

1.2.2 Function Definition

Provides the actual implementation of the function.

```
int add(int a, int b) {  
    return a + b;  
}
```

Declaration is useful when the function is defined later in the code or in a separate file.

1.3 Calling a Function

Once a function is defined, you can use it by simply calling its name followed by parentheses. This tells the program to execute the code inside that function.

If the function takes parameters, you pass the required values inside the parentheses. If it doesn't take any parameters, you just use empty parentheses.

```
#include <iostream>  
using namespace std;
```

```
// Function with no parameters and no return value  
void greet() {  
    cout << "Welcome to C++ Programming!" << endl;
```

```

}

// Function with parameters and a return value
int multiply(int a, int b) {
    return a * b;
}

int main() {

    // Calling the greet function
    greet();

    // Calling the multiply function
    int result = multiply(4, 5);
    cout << "Multiplication result: " << result << endl;

    return 0;
}

```

2. Types of Functions

In C++, functions can be broadly categorized based on two criteria:

2.1. Based on origin

Library Functions: These are built-in functions provided by C++ standard libraries, such as `cout`, `sqrt()`, `abs()`, and `getline()`. You can use them by including appropriate headers like `<iostream>`, `<cmath>`, or `<string>`.

User-Defined Functions: These are functions created by the programmer to perform specific tasks in the program.

2.2. Based on input and return type

User-defined functions can be further classified based on whether they accept parameters or return a value:

No parameters, no return value: The function performs a task but does not take input or return anything.

- I. Parameters, no return value: The function takes input but does not return a result.
- II. No parameters, return value: The function returns a result but does not take any input.
- III. Parameters and return value: The function takes input and returns a result.

3. Parameters or Arguments

A function can also take some input data to process. These values are called function arguments and are supplied to the function at the function call. To receive these values, we define placeholder variables called parameter inside parenthesis in function definition. They should be specified with their types and names.

```
#include <iostream>
using namespace std;

// Defining function that prints given number
void printNum(int n){
    cout << n << endl;
}

int main() {
    int num1 = 10;
    int num2 = 99;

    // Calling printNum and passing both
    // num1 and num2 to it one by one
    printNum(num1);
    printNum(num2);
    return 0;
}
```

4. Function Call By Value

The call-by-value method allows you to copy the actual parameter to a formal parameter. In this case, if we change the formal parameter then the actual parameter doesn't change.

4.1 Use of Call by Value

Call by value is used in certain conditions like:

- I. When you do not want to change the actual parameters of the function.
- II. When you want to make copies of the data instead of the actual data.
- III. When space is not an issue.
- IV. Usually, when you do not deal with recursion or backtracking.

4.2 Limitations of using Call by Value

Although call by value is so useful for us while programming in C++, it has a few limitations too:

- I. Data passed is then stored in temporary memory
- II. Can't operate over the actual data rather need to work on temporary copy made of the data, because of which changes made in the data in the function is not reflected in main function.
- III. Memory Space required while using string ,array , vector, etc can be huge.
- IV. Tackling Backtracking and recursion can be complex using call by values.

// Swapping using Call by function

#include <iostream>

using namespace std;

// Swap function to demonstrate

// call by value method

void swap(int x, int y)

```
{  
    int t = x;  
    x = y;  
    y = t;  
    cout << "After Swapping in function x: " << x  
        << ", y: " << y << endl;  
}
```

// Driver Code

int main()

```
{  
    int x = 1, y = 2;  
  
    cout << "Before Swapping: ";  
    cout << "x: " << x << ", y: " << y << endl;  
  
    swap(x, y);  
  
    cout << "After Swapping: ";  
    cout << "x: " << x << ", y: " << y << endl;  
  
    return 0;  
}
```

5. Call by Reference

When a variable is passed as a reference to a function, the address of the variable is stored in a reference variable inside the function. Hence, the variable inside the function is an alias for the passed variable. Therefore, any operations performed on the variable inside the function will also be reflected in the calling function.

This ability to reflect changes could return more than one value by a function.

Also, a void function could technically return values using this method.

```
#include <iostream>
using namespace std;
```

```
void func(int& x) {
    x--;
}
```

```
int main() {
    int a = 5;
    cout << a << endl;
    func(a);
    cout << a;
}
```

6. Difference between Call By Value and Call By Reference:

<u>Call by Value</u>	<u>Call by Reference</u>
The actual reference (memory address) of the variable is passed.	A copy of the actual value is passed to the function.
The original variable is modified.	The original variable is not modified.
More memory efficient since no copy of the variable is made.	Requires more memory since a copy of the variable is created
Changes inside the function affect the original variable.	Changes inside the function do not affect the original variable.
Suitable when you want to modify the original data or avoid copying large data.	Suitable when you don't want to alter the original data.

7. Recursion

Recursion is a programming technique where a function calls itself repeatedly until a specific base condition is met. A function that performs such self-calling behavior is known as a recursive function, and each instance of the function calling itself is called a recursive call.

```
#include <iostream>
using namespace std;
```

```
void printHello(int n) {
```

```
    // Base Case
```

```

    if (n == 0) return;

    cout << "Hello" << endl;

    printHello(n - 1);
}

int main() {

    printHello(5);

    return 0;

}

```

7.1 Recursive Function

A function that calls itself is called a recursive function. When a recursive function is called, it executes a set of instructions and then calls itself to execute the same set of instructions with a smaller input. A recursive function should contain,

- I. **Recursive Case:** Recursive case is the way in which the recursive call is present in the function.
- II. **Base Condition:** The base condition is the condition that is used to terminate the recursion.

7.2 How Recursion Works?

Each time a function calls itself, the current state is saved on the stack, and the new call begins. Once the base case is reached, the function starts returning back, one call at a time.

7.3 Applications of Recursion

Recursion has many applications in computer science and programming. Here are some of the most common applications of recursion:

- I. **Solving:** Fibonacci sequences, Factorial Function, Reversing an array, Tower of Hanoi.
- II. **Backtracking:** It is a technique for solving problems by trying out different solutions and undoing them if they do not work. Recursive algorithms are often used in backtracking.
- III. **Searching and Sorting Algorithms:** Many searching and sorting algorithms, such as binary search and quicksort, use recursion to divide the problem into smaller sub-problems.

- IV. **Tree and Graph Traversal:** Recursive algorithms are often used to traverse trees and graphs, such as depth-first search and breadth-first search.
- V. **Mathematical Computations:** Recursion is also used in many mathematical computations, such as the factorial function and the Fibonacci sequence.
- VI. **Dynamic Programming:** It is a technique for solving optimization problems by breaking them down into smaller sub-problems. Recursive algorithms are often used in dynamic programming.

Overall, recursion is a powerful and versatile technique that can be used to solve a wide range of problems in programming and computer science.

7.4 Drawbacks of Recursion

- I. **Performance:** Recursive algorithms can be less efficient than iterative algorithms in some cases, particularly if the data structure is large or if the recursion goes too deep.
- II. **Memory usage:** Recursive algorithms can use a lot of memory, particularly if the recursion goes too deep or if the data structure is large. Each recursive call creates a new stack frame on the call stack, which can quickly add up to a significant amount of memory usage.
- III. **Code complexity:** Recursive algorithms can be more complex than iterative algorithms.
- IV. **Debugging:** Recursive algorithms can be more difficult to debug than iterative algorithms, particularly if the recursion goes too deep or if the program is using multiple recursive calls.
- V. **Stack Overflow:** If the recursion goes too deep, it can cause a stack overflow error, which can crash the program.